

Development Plan FPVR

Team 9, Software Mandems
John Cho
Ibrahim Tariq
Vikram Chandar
Sankalpa Chhetri Dhakal
Arjun Sasidaran

Table 1: Revision History

Date	Developer(s)	Change
Sep 27 2026	Vikram Chandar	Add Expected Technology, Coding Standards and Use of AI sections
Sep 27 2026	Sankalpa Chhetri Dhakal	Add IP to Protect section and Project Decomposition and Scheduling sections
Date3	Name(s)	Description of changes
...	...	...

[Put your introductory blurb here. Often the blurb is a brief roadmap of what is contained in the report. —SS]

[Additional information on the development plan can be found in the [lecture slides](#). —SS]

1 Confidential Information?

FPVR is not subject to any form of confidentiality nor should it contain any private interactions or insider information from stakeholders that might be treated as being confidential. Any potentially sensitive information that needs to be recorded will be stored separate from any public repository.

2 IP to Protect

There is currently no intellectual property agreement associated with this project. The project is being developed by the team as part of the capstone course and is not subject to any additional IP protection requirements from an external organization.

3 Copyright License

The team has adopted the MIT License for this project. The MIT License is a short, permissive open-source license that allows anyone to use, modify, and distribute the software, provided that the original copyright notice and license are included. The team chose this license because it is simple to understand and places few restrictions on others who may want to build on the project. The full license is available in the [LICENSE](#) file in the project repository.

4 Team Meeting Plan

The team holds regular internal meetings and also meets with the project supervisor, Dr. Douglas Down.

- **Team meetings:** The team meets once a week, in person, on Thursdays from 3:30 to 4:20 p.m. If the team is unable to meet in person, the meeting is held online on Discord at the same time.

- **Supervisor meetings:** The team meets with Dr. Down in person in his office every two weeks. The exact meeting time will be finalized once a time is confirmed with and approved by the supervisor.
- **Chair and agenda:** Every meeting has a chair and an agenda. The chair posts the agenda in the meeting's GitHub issue at least one day before the meeting.
- **Structure:** Each meeting follows the same structure:
 1. Review of action items from the last meeting.
 2. Progress updates from each member.
 3. Discussion of blockers and decisions.
 4. Assignment of new action items.
- **Notes:** Meeting notes and action items are recorded in a GitHub issue created from the matching meeting template (team meeting or supervisor meeting).

5 Team Communication Plan

The team uses a small set of tools for communication, each with a specific purpose, so that discussions are easy to find and decisions are recorded.

- **Discord:** used for day-to-day messaging, quick questions, and coordinating work. Online team meetings are also held on Discord.
- **GitHub Issues:** used for tasks, bugs, meeting agendas and notes, and decisions, so that there is a written record. Meetings are created from the team's issue templates, and any decision made on Discord is recorded in the relevant issue.
- **Pull request comments:** used for discussion and review of code and documentation changes.
- **External communication:** the team liaison (John Cho) is the main point of contact with the supervisor, TA, and instructor, and shares any updates with the team on Discord.
- **Response time:** members are expected to respond to messages within 24 hours on weekdays.
- **Urgent matters:** handled by mentioning the relevant members directly on Discord, followed by text message if there is no response.

6 Team Member Roles

Each team member has both project-level roles, which support how the team works together, and a technical role, which covers an area of the system they lead.

6.1 Project Roles

- **Team lead (Ibrahim Tariq):** tracks deadlines and keeps the project on schedule.
- **Team liaison (John Cho):** the main point of contact with the supervisor, TA, and instructor.
- **Meeting chair:** sets the agenda and runs the meeting. This role rotates between meetings.
- **Notetaker:** records meeting notes and action items in the meeting's GitHub issue. This role rotates between meetings.
- **Quality assurance:** checks each deliverable against its checklist and the Avenue rubric before submission. This role rotates for each deliverable, and the member who wrote the most of a deliverable does not act as its QA reviewer.
- **Reviewer:** reviews pull requests. Every member reviews, and each pull request needs approval from at least one member other than its author.

6.2 Technical Roles

Each member leads one technical area of the project, as shown in Table 2. The lead is responsible for the design of that area and is the first person to ask about it, but is not the only person who works on it.

Team Member	Technical Area
John Cho	Controller input and hardware integration
Ibrahim Tariq	Flight physics and VR / camera
Sankalpa Chhetri Dhakal	Training environments and ML / AI
Vikram Chandar	Back-end (game state, scoring, flight data)
Arjun Sasidaran	Back-end (game state, scoring, flight data)

Table 2: Technical area leads

6.3 Flexibility

Roles are assigned with overlap and are expected to change throughout the year. Every member contributes to both development and documentation, and work is reassigned based on workload and deadlines rather than strictly by role.

7 Workflow Plan

- **Git:**

- The **main** branch is kept stable and is the source of the published documents.
- Work is done on a separate branch for each issue, named after the issue number (e.g. **feature/18**).
- Changes are merged into **main** through pull requests, which must be reviewed and approved by at least one other team member and pass CI before merging. Each pull request links the issue it resolves (e.g. **Closes #18**) so the issue closes automatically when it is merged.
- After a pull request is merged, its branch is deleted on GitHub and locally. Branches that have had no activity for over two weeks are reviewed at the next team meeting and either finished or deleted, so that the list of branches only shows active work.

- **Tags and releases:**

- When a deliverable is submitted, the commit on **main** that was submitted is marked with an annotated Git tag named after the deliverable and revision (e.g. **DevPlan-Rev0**, **SRS-Rev0**, **Final-Rev1**). This records exactly what was submitted, and makes it easy to compare later revisions against it.
- Tags are created only after the deliverable's pull request has been merged and the quality assurance check is complete.
- Once working software exists, each demonstration (the proof of concept, Revision 0, and Revision 1) is also published as a GitHub Release from its tag, with a short summary of the features included.

- **Issues:**

- Every task, bug, and meeting is tracked as a GitHub issue.
- Issue templates are used for recurring issue types: team meetings, supervisor meetings, TA meetings, lectures, and peer reviews. These templates automatically apply the **meeting** or **review** label.

- Other issues, such as development tasks and bugs, are created as blank issues. These are classified with labels that describe the type of work, such as **documentation**, **enhancement**, and **bug**.
- Each issue is assigned to a team member and tracked on the project's Kanban board.
- **CI/CD:**
 - A GitHub Actions workflow compiles the LaTeX documents on every pull request to **main**, so documents that fail to build are caught before merging.
 - On every push to **main**, the same workflow rebuilds only the documents affected by the change, commits the generated PDFs to the **pdfs/** folder, and deploys them to GitHub Pages for review.
 - Once source code is added, the CI pipeline will be extended to build the project and run the unit tests on every pull request.

8 Project Decomposition and Scheduling

The team will use GitHub Projects to organize and track the development of the project. A Kanban board has been created with the following stages: *Backlog*, *Ready*, *In Progress*, *In Review*, and *Done*. Tasks will move through these stages as work is planned, implemented, reviewed, and completed.

The project is decomposed into smaller GitHub Issues representing development tasks, research activities, meetings, and course deliverables. Labels are used to identify the type of work, such as research, development plan tasks, problem statement tasks, and meetings. This allows the team to assign work to individual members and monitor the progress of each task.

The GitHub Project can be found at:

<https://github.com/orgs/SoftwareMandems/projects/1>

The overall project schedule will follow the major capstone milestones and deadlines provided in the course outline. In addition to course deliverables, technical work will be scheduled in stages. The early stages will focus on setting up the Unity and VR environment and developing a basic FPV drone flight model. Once the core simulation is working, development will move toward controller support, training environments, scoring and feedback systems, multiplayer functionality, and the planned AI-based flight analysis features.

Tasks that are not yet ready to be worked on will remain in the backlog, while tasks that are prepared for development will move into the ready state. Work

currently being completed will be tracked as in progress, and completed work will be reviewed before being moved to done. This workflow will allow the team to continuously track progress and adjust the schedule as priorities change.

9 Proof of Concept Demonstration Plan

What is the main risk, or risks, for the success of your project? What will you demonstrate during your proof of concept demonstration to convince yourself that you will be able to overcome this risk?

10 Expected Technology

The project will use an open source game engine like Unity as the primary game engine and C# as the primary programming language. C# will be used to implement the drone simulation, flight controls, physics, user interfaces, VR interactions, multiplayer functionality, and AI-based flight analysis. Unity and C# were selected because they provide established tools for VR development, physics simulation, and cross-platform deployment. However, Unity's abstractions may limit access to lower-level hardware functionality and may introduce performance overhead for computationally intensive components. C may therefore be investigated for lower-level components such as drivers if direct hardware access or performance requirements make it necessary.

Unity's built-in physics, input, and rendering systems will be used where appropriate to support the drone simulation. These systems reduce the need to implement common functionality from scratch, but their general-purpose implementations may not provide the level of control or performance required for realistic FPV flight. Unity's VR and XR packages such as OpenXR Plugin and XR Interaction Toolkit will be investigated for integration with the project's VR hardware, with compatibility and input latency evaluated during development. Additional Unity packages and libraries such as Unity Multiplayer SDK, Unity Transport and Unity Netcode may be used for multiplayer networking, data collection, visualization, and other functionality required by the project. These packages will be evaluated based on their compatibility, performance, documentation, and ability to meet the project's requirements, with custom implementations considered where existing functionality is insufficient.

For the AI-based flight analysis component, pre trained models are expected to be leveraged rather than developing a model from scratch. Open Source models like Qwen2-VL may be investigated to provide feedback on areas such as control efficiency and flight performance. However, general-purpose models may not re-

liably interpret the structured controller inputs and flight telemetry required by the project and may produce inconsistent feedback. Existing models and libraries will therefore be tested against representative flight data to determine whether they satisfy the project's accuracy, performance, and hardware requirements before considering fine-tuning, adapting an existing model, or developing a custom model.

The project can use a standard linter such as Microsoft C# linter and formatter to maintain consistent coding conventions and identify potential issues during development. The linter will provide automated checks for common coding problems, although static analysis cannot detect all runtime or simulation-specific issues. Unit testing will therefore be performed using an appropriate C# testing framework, such as NUnit or Unity's Test Framework. Code coverage tools will also be investigated to measure the extent to which automated tests exercise the implemented functionality and identify areas requiring additional tests.

Continuous Integration (CI) will be implemented using GitHub Actions where practical. Automated builds, tests, and code-quality checks will be run when changes are pushed to the repository or submitted through pull requests. GitHub Actions was selected because it integrates with the team's existing GitHub workflow and can automate repetitive verification tasks. However, Unity projects can require significant build dependencies and execution time, so the CI workflow will be limited to checks that can be reliably automated within the available resources. This will help identify integration and testing issues before changes are merged into the main branch.

Git, GitHub, and GitHub Projects will be used for version control, collaboration, issue tracking, and project management. GitHub Issues will be used to track development tasks and defects, while GitHub Projects will be used to organize and monitor the project's overall progress. These tools provide an integrated workflow for coordinating development, while working within the github ecosystem.

The team will use an open source IDE like Visual Studio Code as the primary IDE for development, debugging, and integration with Unity. Visual Studio Code provides a lightweight development environment and supports the project's required languages and tools, although its Unity and C# functionality may not provide the same level of integrated tooling as a full-featured IDE but it gives familiarity and flexibility if changes are to be made in the future. Code coverage can be measured using a separate Unity's Code Coverage package to determine the percentage of C# code executed by the project's automated tests.

The team will use a free performance profiler such as the Unity Profiler to measure CPU, GPU, memory, physics, and rendering performance throughout

development. The Unity Memory Profiler and Frame Debugger will also be investigated to identify memory usage issues and rendering bottlenecks, particularly for maintaining the performance requirements of the VR simulation. These tools provide detailed information about Unity’s runtime behaviour, but profiling results may not fully represent performance on the final VR hardware. Performance testing will therefore also be performed on the target hardware to validate that optimizations identified through profiling translate into acceptable real-world performance.

Although existing Unity systems, libraries, and frameworks will be used where appropriate, major project-specific components such as the drone flight model, controller behaviour, flight evaluation system, and integration between the simulation, VR controls, multiplayer functionality, and AI analysis will be implemented through custom code. Existing tools will be preferred when they provide reliable and maintainable functionality, while custom implementations will be considered when their limitations prevent them from meeting the project’s requirements.

11 Use of AI and LLMs

The team will use Claude Code and ChatGPT as development and documentation assistance tools throughout the project. These tools may be used to assist with generating code, explaining unfamiliar concepts, debugging, developing test cases, reviewing code, and drafting or improving project documentation.

LLM-generated code and documentation will not be treated as the standard. All generated code will be reviewed and tested by team members before being incorporated into the project. The team will verify functionality through unit tests, integration testing, manual testing, and performance testing where appropriate. Documentation generated with the assistance of LLMs will also be reviewed by team members against project requirements and rubrics to ensure that it is accurate.

12 Coding Standard

The project will use the **Microsoft C# Coding Conventions** as the coding standard for all C# source code. This will provide consistent conventions for naming, formatting, organization, and documentation throughout the project.

PascalCase will be used for classes, methods, and public properties, and camel-Case for local variables and parameters. Methods and classes will have descrip-

tive names, and code will be organized into modular classes with each class having a clear responsibility.

The project will use consistent indentation and brace formatting, avoid unnecessary code duplication, and include comments where they help explain non-obvious implementation details. Code will be formatted consistently using the development environment's formatting tools. Pull requests will be reviewed by another team member before being merged into the main branch to ensure the coding standard is followed.

13 Appendix — Generative AI Use Disclosure

13.1 AI Tools Used

- ChatGPT 5.6 Sol, OpenAI

13.2 How and Where Used

ChatGPT 5.6 Sol was used only to review the document for grammar, spelling and other minor phrasing issues. It was not used to generate any technical content, project ideas, requirements or diagrams.

Grammar checking was performed throughout the document where necessary to improve readability and correct grammar errors. None of the 13 sections of this document were generated or directly altered by the AI tool.

13.3 Verification of AI-Generated Content

All grammar and spelling corrections that were suggested by ChatGPT 5.6 Sol, were reviewed and added into this document manually. The team ensured that grammatical corrections did not alter the intended technical meaning or introduce new factual or technical claims.

The team remains responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, as no content was directly generated by an AI system.

Appendix — Reflection

Sankalpa - Reflection

Why is it important to create a development plan prior to starting the project?

Creating a development plan before starting the project is important because it gives the team a shared understanding of how the project will be organized and developed. Without a plan, it would be easy for team members to have different expectations about responsibilities, workflows, deadlines, and development practices. While working on the Project Decomposition and Scheduling section, I found that breaking the project into GitHub Issues and organizing them on a Kanban board made the overall project feel much more manageable. Instead of viewing the FPV drone simulator as one large project, we can approach it as smaller tasks that can be assigned, tracked, reviewed, and completed individually. I also think the plan will be useful later in the year because we now have an initial structure that we can adjust as our priorities and understanding of the project change.

In your opinion, what are the advantages and disadvantages of using CI/CD?

One of the biggest advantages of CI/CD is that it can catch problems early. Automated builds, tests, and other checks can run whenever changes are pushed or a pull request is created, which reduces the chance of broken or incompatible code being merged into the main branch. This is especially useful for a team project because multiple people will be developing different parts of the system at the same time. It also encourages the team to integrate changes frequently instead of waiting until large amounts of code have been written separately.

A disadvantage is that CI/CD requires additional setup and maintenance. For our project, which involves Unity and VR, configuring automated builds and tests may be more complicated than it would be for a simpler software project. A CI pipeline can also become frustrating if it is poorly configured and fails for reasons unrelated to the actual code. Because of this, I think CI/CD is valuable for our project, but we should automate checks where they provide a clear benefit rather than adding automation simply for the sake of having it.

What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Our group did not have any major disagreements while completing this deliverable. Most discussions were about how we wanted to organize the project and how much detail we should include at this early stage. For example, while setting up the project decomposition and scheduling, we had to decide how tasks should be represented and tracked through GitHub Issues and the Kanban board. We resolved these smaller differences through discussion and by choosing approaches that were simple enough to use now but could still be expanded later as the project becomes more complex. Overall, responsibilities were divided between team members, and we reviewed the combined work to make sure the different sections were consistent with the direction of the project.

Appendix — Team Charter

[borrows from [University of Portland Team Charter](#) —SS]

External Goals

[What are your team’s external goals for this project? These are not the goals related to the functionality or quality of the project. These are the goals on what the team wishes to achieve with the project. Potential goals are to win a prize at the Capstone EXPO, or to have something to talk about in interviews, or to get an A+, etc. —SS]

Attendance

Expectations

[What are your team’s expectations regarding meeting attendance (being on time, leaving early, missing meetings, etc.)? —SS]

Acceptable Excuse

[What constitutes an acceptable excuse for missing a meeting or a deadline? What types of excuses will not be considered acceptable? —SS]

In Case of Emergency

[What process will team members follow if they have an emergency and cannot attend a team meeting or complete their individual work promised for a team deliverable? —SS]

Accountability and Teamwork

Quality

[What are your team’s expectations regarding the quality of team members’ preparation for team meetings and the quality of the deliverables that members bring to the team? —SS]

Attitude

[What are your team's expectations regarding team members' ideas, interactions with the team, cooperation, attitudes, and anything else regarding team member contributions? Do you want to introduce a code of conduct? Do you want a conflict resolution plan? Can adopt existing codes of conduct. —SS]

Stay on Track

[What methods will be used to keep the team on track? How will your team ensure that members contribute as expected to the team and that the team performs as expected? How will your team reward members who do well and manage members whose performance is below expectations? What are the consequences for someone not contributing their fair share? —SS]

[You may wish to use the project management metrics collected for the TA and instructor for this. —SS]

[You can set target metrics for attendance, commits, etc. What are the consequences if someone doesn't hit their targets? Do they need to bring the coffee to the next team meeting? Does the team need to make an appointment with their TA, or the instructor? Are there incentives for reaching targets early? —SS]

Team Building

[How will you build team cohesion (fun time, group rituals, etc.)? —SS]

Decision Making

[How will you make decisions in your group? Consensus? Vote? How will you handle disagreements? —SS]